

Programmation en C

John Samuel

CPE Lyon

Année: 2025-2026

Courriel: john.samuel@cpe.fr



THE C PROGRAMMING LANGUAGE



- **Enseignant-Chercheur**, Conception Logicielle et Big Data, CPE Lyon,
- **Intérêts et thèmes de recherche** :
Représentation de connaissances, le web sémantique, les services web, l'intégration de données, l'entrepôt de données, les systèmes distribués, système d'information géographique
- **Cours** : Programmation en C, Algorithmes en C, Data Mining et Machine Learning, Intelligence Artificielle et Deep Learning, Systèmes d'exploitation et Programmation Concurrente, Langages Web
- **Thèse** : Intégration des données issues de services web

Objectifs



- Comprendre les **bases de la programmation** en C, y compris l'histoire et la compilation.
- Maîtriser les concepts de base tels que les **types de données, les variables, et les opérateurs**.
- Savoir utiliser les **structures de contrôle** pour créer des branchements conditionnels et des boucles.
- Explorer les concepts avancés liés aux **pointeurs, aux structures, et aux fonctions**.
- Apprendre la **gestion de la mémoire**, la manipulation de fichiers, et l'allocation dynamique de mémoire.
- Comprendre les **appels systèmes**, la gestion des erreurs, et la création de chaînes de compilation.
- Se familiariser avec des sujets **avancés** tels que l'architecture **client-serveur**, la manipulation de fichiers, et la création de **Makefiles**.

1.1. Programmation en C: Introduction

Bonjour le Monde!!

```
/* Fichier: bonjour1.c
 * affiche 'Bonjour le Monde!!!' à l'écran.
 * auteur: John Samuel
 * Ceci est un commentaire sur plusieurs lignes
 */

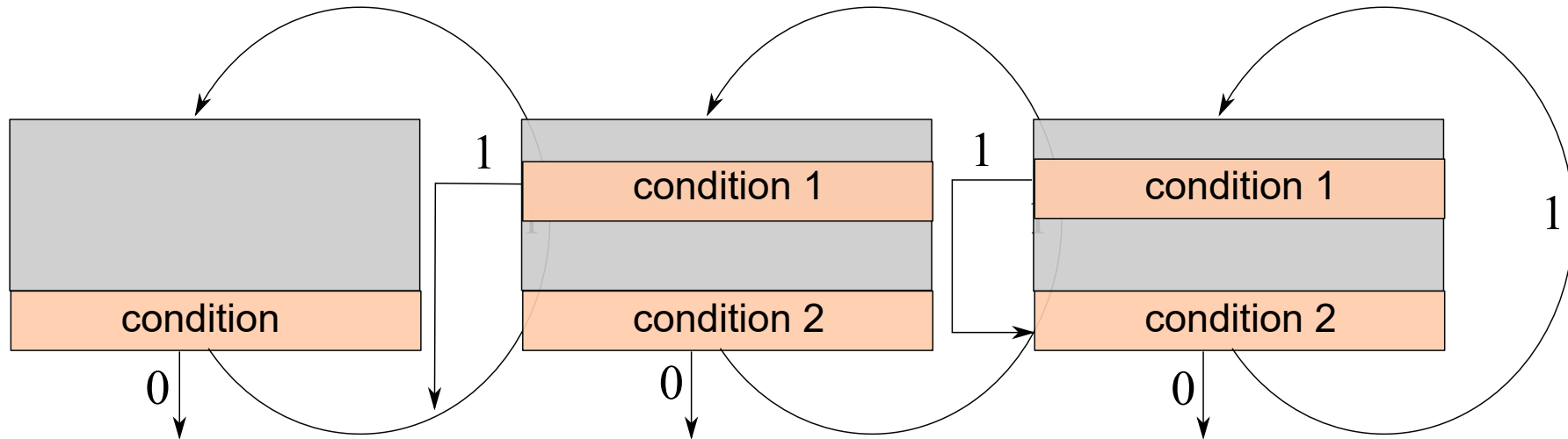
#include <stdio.h> // headers

// Ceci est un commentaire sur une ligne

int main() {
    printf("Bonjour le Monde !!!");
    return 0;
}
```

1.1. Programmation en C: Introduction

Structures de contrôle



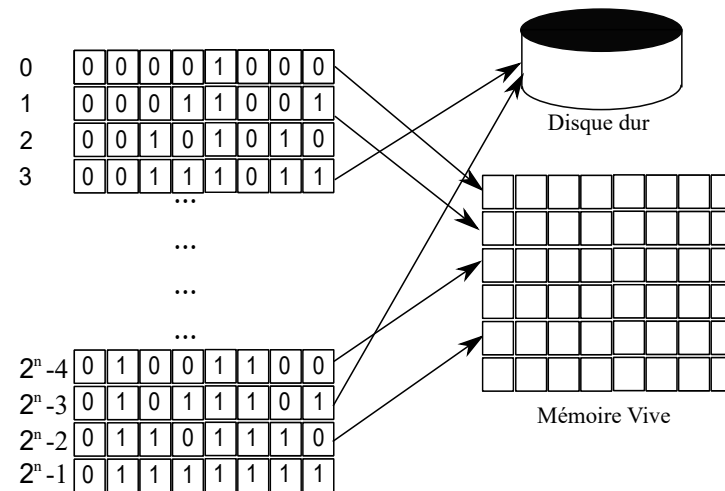
Les structures de contrôle en C dirigent l'exécution du code en fonction de conditions ou de boucles.

1.1. Programmation en C: Introduction

Gestion de mémoire

0	0	0	0	0	1	0	0	0
1	0	0	0	1	1	0	0	1
2	0	0	1	0	1	0	1	0
3	0	0	1	1	1	0	1	1
...								
...								
...								
...								
2^n-4	0	1	0	0	1	1	0	0
2^n-3	0	1	0	1	1	1	0	1
2^n-2	0	1	1	0	1	1	1	0
2^n-1	0	1	1	1	1	1	1	1

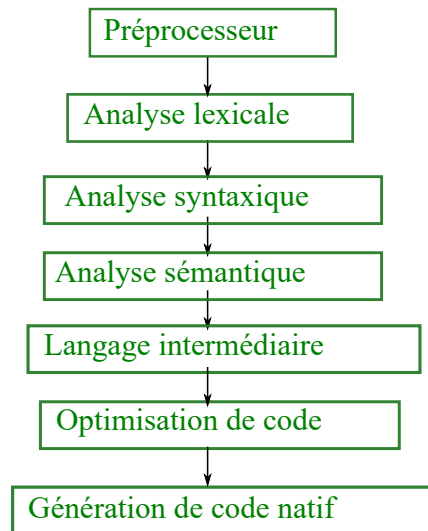
Adresses physiques



La conversion des adresses virtuelles en adresses physiques

1.1. Programmation en C: Introduction

La chaîne de compilation

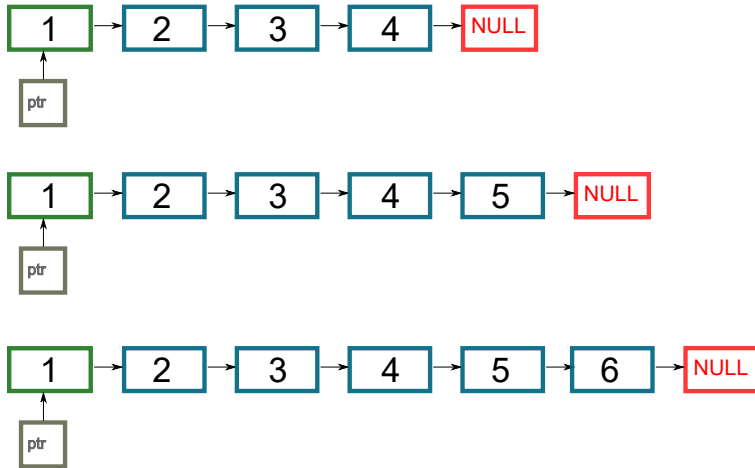


Compilation: gcc

La chaîne de compilation en C est le processus séquentiel de **transformation du code source** en un exécutable.

1.1. Programmation en C: Introduction

File

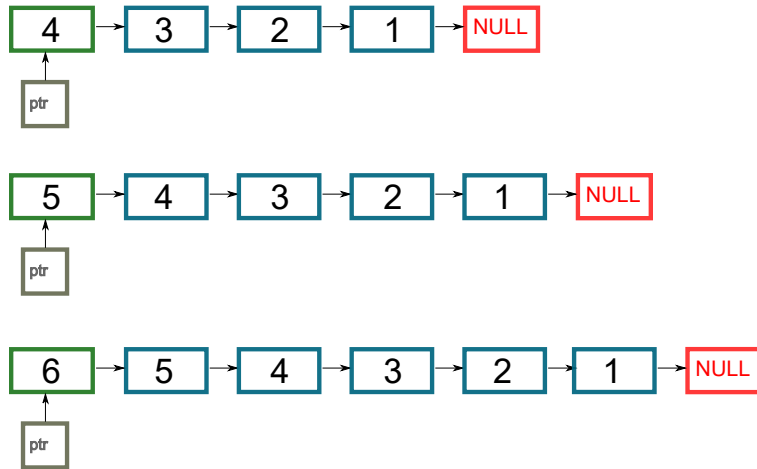


File

Une file est une structure de données qui gère les éléments de manière **FIFO (First-In-First-Out)**, ce qui signifie que l'élément ajouté en premier est le premier à être retiré.

1.1. Programmation en C: Introduction

Pile



Pile

Une pile est une structure de données qui gère les éléments de manière **LIFO (Last-In-First-Out)**, où le dernier élément ajouté est le premier à être retiré.

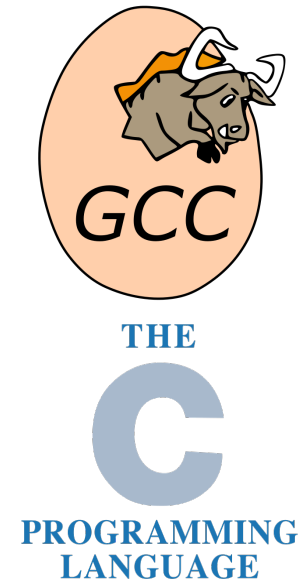
1.1. Programmation en C: Introduction

Environnement de programmation:

- Système d'exploitation: Linux
 - Ubuntu
 - Machine virtuelle (VirtualBox) + Ubuntu
- Compilateur: gcc
- Éditeur: Vim, Emacs (ou selon votre choix)

Cours:

- Interactifs
- Programmation en live
- Référence: <https://github.com/johnsamuelwrites/jscourses>
- Les questions: chaque 20-30 mins
- Devoir surveillé: 60%



1.1. Programmation en C: Introduction

Devoir surveillé (DS): 60%

- **Examen:** En-ligne sur E-campus
- **Durée:** 2 heures
- **Total:** 20 points
- **Documents:** autorisés
- **Types de documents autorisés:** Tous les documents autorisés
- **Calculatrices :** non autorisées
- **Utilisation de l'internet :** non autorisée
- **Dépôt supplémentaire :** disponible pour les fichiers personnels

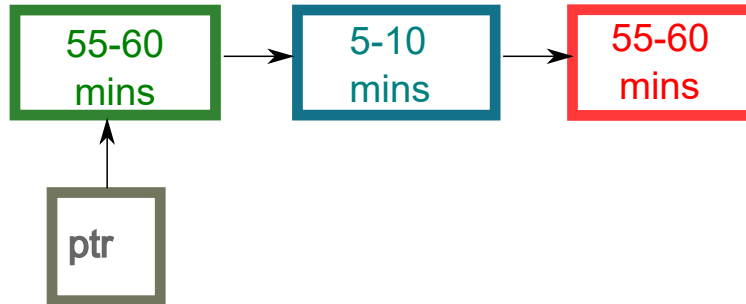
Vous recevrez un courrier détaillé avant l'examen

1.1. Programmation en C: Introduction

Cours	Dates
Cours 1	9 septembre
Cours 1	11 septembre
Cours 2	16 septembre
Cours 3	18 septembre
Cours 4	24 septembre
Cours 5	25 septembre

1.1. Programmation en C: Introduction

Déroulement du cours



Le cours de 2 heures est structuré avec une **première session** de 55 à 60 minutes, suivie d'une **pause** de 5 à 10 minutes pour des questions, et conclut ensuite avec une **deuxième session** de 55 à 60 minutes.

1.1. Programmation en C: Introduction

Travaux pratiques, projet et évaluation

- Les 6 travaux pratiques (TP) seront **évalués**. L'évaluation est divisée en deux parties distinctes : **TP1 à TP3** d'un côté et **TP4 à TP6** de l'autre.
- Deux **dates limites** de soumission sont précisées sur e-campus, correspondant à chaque partie de TP.
- Il y a 3 **groupes**. Chaque groupe bénéficiera de l'encadrement **d'un intervenant**.
- Il est fortement conseillé de **travailler en binôme** pour favoriser la collaboration et l'efficacité.
- **Évaluation** : deux parties :
 1. Évaluation des rendus de TP (**évaluation du travail du groupe**)
 2. **Évaluation individuelle** (dernière séance)
- **L'évaluation individuelle** porte sur :
 - Les **travaux réalisés** et leur compréhension
 - Le **langage C**
 - Les réflexions sur **le rôle et la valeur d'un développeur C dans un monde avec l'IA et les chatbots LLMs**.

Évaluation des TP

TP	Points (sur 20)
TP 1 (évaluation du travail du groupe)	2
TP 2 (évaluation du travail du groupe)	2
TP 3 (évaluation du travail du groupe)	3
TP 4 (évaluation du travail du groupe)	2
TP 5 (évaluation du travail du groupe)	3
TP 6 (évaluation du travail du groupe)	3
Évaluation individuelle	5

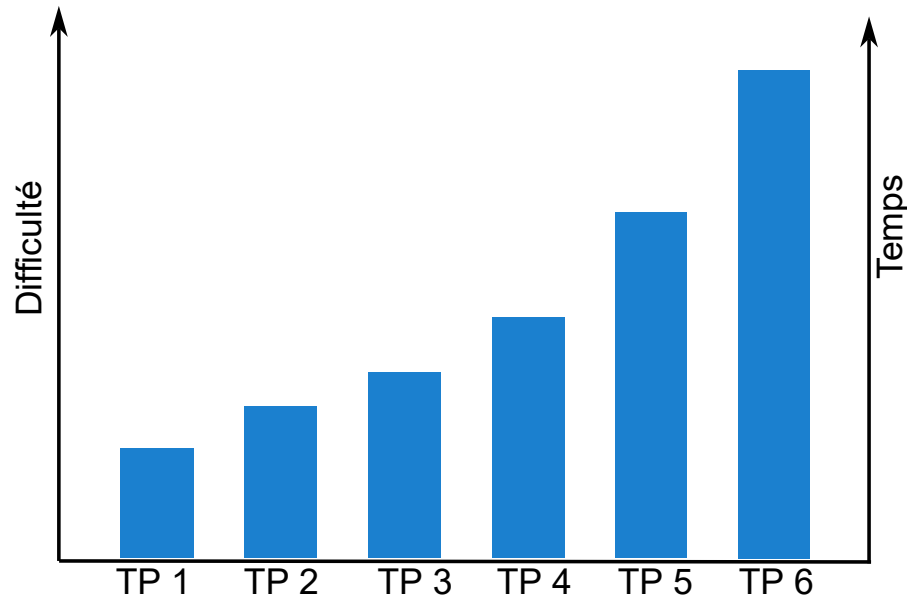
Travaux pratiques

Chaque TP comporte plusieurs exercices. Chaque exercice est accompagné d'une indication de **niveau de difficulté** :

- ★: Facile
- ★★: Difficulté moyenne
- ★★★: Difficile

1.1. Programmation en C: Introduction

Travaux pratiques et Projet



La **complexité** des TP augmente progressivement après chaque séance, ce qui peut nécessiter plus de **temps** pour compléter tous les exercices.

Travaux pratiques: Rendus

1. Le rendu est constitué de deux parties principales : le rapport d'auto-évaluation et les fichiers source.
2. Votre dossier de rendu doit comporter les éléments suivants
 - Un fichier **README.md** contenant le rapport d'auto-évaluation.
 - Un fichier **CONTRIBUTORS.md** listant les noms et prénoms des contributeurs.
 - Un répertoire **src/** contenant les fichiers sources, identifiables par les extensions **.c** et **.h**.

1.1. Programmation en C: Introduction

Travaux pratiques: Rendus

1. Votre rendu doit être renommé au format "groupe_N1_N2.zip", où N1 et N2 représentent les noms (par exemple, "groupe_DUPONT_SMITH.zip").
2. Assurez-vous d'écrire ou de compléter les fichiers README.md et CONTRIBUTORS.md au format Markdown.
3. Le contenu du fichier README.md devrait comprendre :
 - i. **Rapport d'auto-évaluation** pour le TP numéro N (où N varie de 1 à 6).
 - ii. **Bibliothèques** utilisées, y compris les bibliothèques standard.
 - iii. **Références** telles que les URL ou les groupes consultés pour l'exercice.
 - iv. Indication du **niveau de difficulté** (facile, moyen, difficile).
 - v. Optionnellement, des **commentaires** ou des remarques supplémentaires peuvent être inclus.

Liste de contrôle

Avant de soumettre votre travail pratique, veuillez vérifier que vous avez respecté la liste de contrôle suivante :

-  Les noms complets (prénom et noms) de la binôme sont correctement inclus dans le fichier CONTRIBUTORS.md.
-  Le fichier README.md est rempli de manière exhaustive et conforme aux instructions fournies.
-  Vous avez respecté les noms de fichiers tels que spécifiés dans chaque exercice des travaux pratiques.
-  Votre code est accompagné de commentaires appropriés pour expliquer la logique et la fonctionnalité.
-  Votre code peut être compilé sans générer d'erreurs, et dans la mesure du possible, sans générer d'avertissements.

1.1. Programmation en C: Introduction

Modèle de code

Vous pouvez consulter <https://github.com/johnsamuelwrites/ProgC> en ligne ou le cloner sur votre machine à l'aide du terminal en utilisant les commandes suivantes.

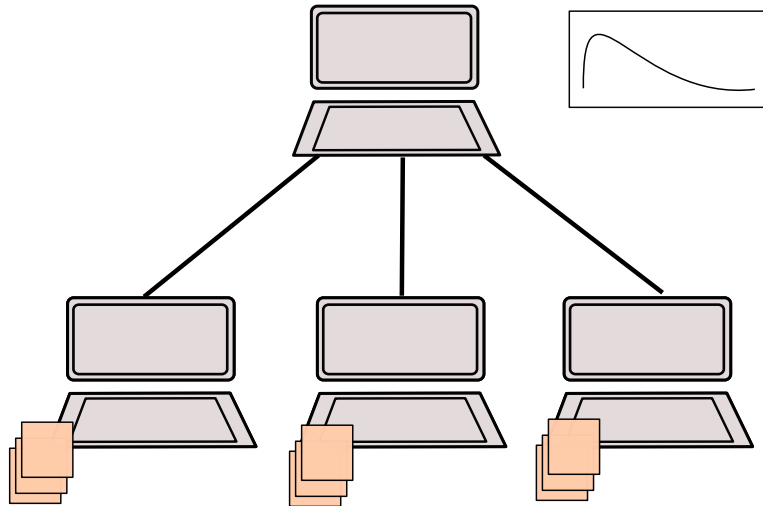
```
$ git clone https://github.com/johnsamuelwrites/ProgC  
$ cd ProgC  
$ ls
```

Pour obtenir les dernières mises à jour du répertoire, vous pouvez exécuter la commande suivante:

```
$ git pull
```

1.1. Programmation en C: Introduction

Travaux pratiques et Projet

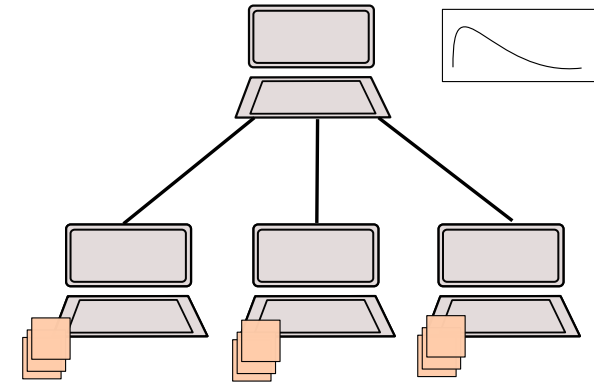


- Le projet est basé sur une architecture client-serveur.
- Les clients sont chargés de réaliser des analyses d'images.
- Contrairement à la méthode classique, les clients ne transmettent pas les images complètes au serveur.
- Au lieu de cela, ils envoient uniquement les résultats d'analyse générés localement.

1.1. Programmation en C: Introduction

Travaux pratiques et Projet

- Ce modèle est similaire au concept d'"**edge computing**".
- L'objectif est d'optimiser l'utilisation de la bande passante et de réduire la charge du serveur.
- Il garantit également la **confidentialité des données** d'image.
- Le serveur agrège les résultats d'analyse de plusieurs clients pour obtenir une vue globale ou des informations agrégées.
- Cette approche est particulièrement efficace pour des applications impliquant la surveillance de nombreux capteurs, la collecte de données distribuées et la gestion de ressources informatiques décentralisées.



1.1. Programmation en C: Introduction

Objectifs des travaux pratiques

1. Comprendre la chaîne de compilation en utilisant **GCC (GNU compilation chain)** :
prétraitement jusqu'à l'exécution
2. Travailler avec les bibliothèques standards et les appels de système d'exploitation (fichiers, les sockets réseau etc.)
3. Comprendre les pointeurs et manipuler les buffers en utilisant les pointeurs
4. Créer, compiler et exécuter un projet C
5. Travailler avec les différents formats de données comme BMP
6. Créer un simple programme client-serveur pour le transfert de fichiers et messages
7. Exécuter les applications existantes sur la machine (e.g. GNU plot)
8. Documenter une application en utilisant le format markdown (*.md)
9. Tester une application

1.1. Programmation en C: Introduction

Attention

1. Aucune soumission par courriel électronique.
2. Les liens vers des sites externes comme GitHub/GitLab,... ne seront pas acceptés.
3. Seules les soumissions complètes sur E-Campus seront acceptées pour l'évaluation.
4. Ne pas modifier les noms de fichiers (*.c, *.h, *.md, ...).

1. Programmation en C: Références

Références

- Langage C, Claude Delannoy
- [https://fr.wikipedia.org/wiki/C_\(langage\)](https://fr.wikipedia.org/wiki/C_(langage))
- https://en.wikipedia.org/wiki/C_data_types

Crédits d'images

- [Wikimedia Commons](#)